PapaBear Partner App

Product User API Migration Plan

Generated: 2026-10-26

Rule: For product users, swap endpoint paths only. **Do not change request params / body field names.** App selects path with **isProductUser**.

Base URL: https://adminpapa.papabear4u.com/
All paths below are relative under that host.

Implementation order

1) Offers 2) Limited Offers 3) Ads / Gallery 4) Withdrawals verify 5) Shared modules confirm

1. Module checklist (product user)

Offers	api/v1/offers/* + getPartnerServices	YES (services → products)	Migrate
Limited Offer	api/v1/timed_offers/*	YES	Migrate
Gallery	salon/individual update + uploadImage	YES if product-scoped	Migrate
Publish / Manage Ads	ads/* , partner-ads/*	YES if product-scoped	Migrate
Withdrawals	withdrawals/*	Confirm product wallet	Verify / Migrate
History	product_history/*	Already product	Keep
Products CRUD	products/* , product/*	Already product	Keep
Auth (product)	product/* , product_user/*	Already product	Keep
Profile / Premium / Notify / Locale	shared	Confirm or keep shared	Confirm
Holidays / Facilities / Slots / Services / Stylist / Packages / Appointment History	service booking APIs	N/A — hidden for product user	Skip

Flow: UI → Controller → Parser → if isProductUser then product_* path else salon path. Body unchanged.

2. Offers (highest priority — uses service_ids)

Current app files: coupons_parse.dart, coupon_controller.dart, coupons_model.dart. Create/update body includes **service_ids**. Product plan: keep field name **service_ids**; backend treats values as product IDs for product users. Partner picker swaps getPartnerServices → getPartnerProducts (same body { uid }).

2.1 Endpoint map

List by user	api/v1/offers/getCouponByUserId	api/v1/product_offers/getCouponByUserId	{ uid }	POST
Get by id	api/v1/offers/getById	api/v1/product_offers/getById	{ id, uid }	POST
Create	api/v1/offers/create	api/v1/product_offers/create	see 2.2	POST / multipart
Update	api/v1/offers/update	api/v1/product_offers/update	see 2.2 + id	POST / multipart
Destroy	api/v1/offers/destroy	api/v1/product_offers/destroy	{ id, uid }	POST
Update status	api/v1/offers/updateStatus	api/v1/product_offers/updateStatus	{ id, uid, status }	POST
Get all	api/v1/offers/getAll	api/v1/product_offers/getAll	token	GET
Get active	api/v1/offers/getActive	api/v1/product_offers/getActive	token	GET
Get stores	api/v1/offers/getStores	api/v1/product_offers/getStores	{ uid }	POST
Partner items	api/v1/offers/getPartnerServices	api/v1/product_offers/getPartnerProducts	{ uid }	POST
List for offers	api/v1/salon/getListForOffers	api/v1/product_offers/getListForOffers	token	GET

2.2 Create / Update body fields (unchanged)

uid, name, short_descriptions, code, type, discount, upto, start_date, expire, max_usage, min_cart_value, **service_ids** (array of IDs — product IDs for product user). Optional image multipart.

NOTE: Do not rename service_ids → product_ids in v1. Param contract stays identical.

3. Limited Offers (timed_offers)

Files: timed_offers_parse.dart, timed_offers_controller.dart. Partner services currently reuse offers/getPartnerServices. For product user: product_timed_offers/* + product_offers/getPartnerProducts.

Campaigns	api/v1/timed_offers/getPartnerCampaigns	api/v1/product_timed_offers/getPartnerCampaigns	{ uid }	POST
Partner create	api/v1/timed_offers/partnerCreate	api/v1/product_timed_offers/partnerCreate	campaign body + uid	POST / multipart
Partner update	api/v1/timed_offers/partnerUpdate	api/v1/product_timed_offers/partnerUpdate	campaign body + uid	POST / multipart
Partner items	api/v1/timed_offers/getPartnerItems	api/v1/product_timed_offers/getPartnerItems	{ uid, campaign_id }	POST
Partner offer	api/v1/timed_offers/getPartnerOffer	api/v1/product_timed_offers/getPartnerOffer	uid, campaign_id?	GET/POST
Update item	api/v1/timed_offers/updateItem	api/v1/product_timed_offers/updateItem	item body	POST
Remove item	api/v1/timed_offers/removeItem	api/v1/product_timed_offers/removeItem	item body	POST
Partner items list	api/v1/offers/getPartnerServices	api/v1/product_offers/getPartnerProducts	{ uid }	POST

4. Ads (Publish / Manage)

Create (legacy)	api/v1/ads/create	api/v1/product_ads/create	ad body + uid	POST
Create partner	api/v1/partner-ads/create	api/v1/product_partner_ads/create	ad body + uid	POST
List added	api/v1/ads/getAllAdded	api/v1/product_ads/getAllAdded	{ uid }	POST
Update	api/v1/ads/update	api/v1/product_ads/update	ad body	POST
Destroy	api/v1/ads/destroy	api/v1/product_ads/destroy	{ id, uid }	POST
Home ads	api/v1/partner-ads/getHome	api/v1/product_partner_ads/getHome	uid query	GET
All banners	api/v1/partner-ads/getAll	api/v1/product_partner_ads/getAll	token	GET/POST
Upload image	api/v1/uploadImage	api/v1/product/uploadImage (or keep shared)	multipart image	POST

5. Gallery

Get profile (salon)	api/v1/salon/getById	api/v1/product_user/getById	{ id }	POST
Update gallery (salon)	api/v1/salon/update	api/v1/product_user/update	profile body + images	POST
Upload image	api/v1/uploadImage	api/v1/product/uploadImage	multipart	POST

6. Withdrawals

Create	api/v1/withdrawals/create	api/v1/product_withdrawals/create	withdrawal body + uid	POST
History	api/v1/withdrawals/history	api/v1/product_withdrawals/history	{ uid }	POST
Filter by date	api/v1/withdrawals/filter	api/v1/product_withdrawals/filter	{ uid, start, end }	POST

If backend confirms shared wallet APIs already support type=product, keep current paths.

7. Already product — keep (do not rebuild)

7.1 Auth

Signup form	api/v1/product/getSignupForm	product_user/getSignupForm	Done
Signup	api/v1/product/signup	product_user/signup|register	Done
Login	api/v1/product/login	product_user/login	Done
Phone login	api/v1/product/loginWithPhonePassword	product_user/...	Done
OTP login	api/v1/product/loginWithMobileOtp	product_user/...	Done
Forgot / reset	api/v1/product/forgotPassword	aliases listed in app	Done
Upload	api/v1/product/uploadImage	product_user/uploadImage	Done

7.2 Products CRUD / stock / history

List seller	api/v1/products/getSellerList	getList	uid / seller
Create	api/v1/products/create	product/create, product_user/create	product body
Create many	api/v1/products/createMultiple	product/createMultiple	array
Get by id	api/v1/products/getById	product/getById	{ id }
Update	api/v1/products/update	product/update	product body
Destroy	api/v1/products/destroy	product/destroy	{ id }
Add stock	api/v1/products/addStock	product/addStock	stock body
Units	api/v1/units/getAll	shared	—
History by product	api/v1/product_history/getByProduct	products/getProductHistory	product id
My order list	api/v1/product_history/getMyList	seller history	seller uid
Order info	api/v1/product_order/getInfoOwner	product orders	order id
Order update	api/v1/product_order/update	status etc	order body
Order stats	api/v1/product_order/getStats	months/all variants	uid

8. Shared modules (confirm with backend)

Profile get	api/v1/profile/getByID	Keep or product_user/getById	{ id }
Profile update FCM	api/v1/profile/update	Keep shared	token body
Owner info	api/v1/profile/getOwnerInfo	Keep shared	uid
Tax availability	api/v1/pricing/getTaxAvailability	Keep shared	country
Upgrade plans	api/v1/upgrade/getPlans|plans	Keep or product upgrade	—
Payment link	api/v1/upgrade/generatePaymentUrl	Keep shared	plan body
Notifications	api/v1/notifications/all|read	Keep shared	uid
Locale	api/v1/locale/*	Keep shared	lang
Settings	api/v1/settings/getDefault	Keep shared	—
Cities / countries	api/v1/cities/*	Keep shared	country
Complaints	api/v1/complaints/*	Keep shared	uid
Chat	api/v1/chats/*	Keep shared	uid
Contact us	api/v1/contacts/create	Keep shared	message

9. App wiring plan (after backend confirms paths)

1. Add product_* constants in lib/app/util/constants.dart next to salon paths.
2. CouponsParser / TimedOffersParser / Ads*Parser / GallaryParser / WithdrawalsParser: select path via ProductUser.isProduct(sharedPreferencesManager).
3. Do not change request body maps in controllers.
4. Offers UI can label picker as Products for product users; still posts service_ids.
5. Salon/individual behavior unchanged.

10. Success criteria

• Backend implements product_offers/* and product_timed_offers/* with identical bodies.
• getPartnerProducts returns product rows shaped like service rows (id, name, price, status…).
• App endpoint swap by isProductUser only.
• No rename of service_ids in client v1.